

Frontend: Theory Assignment 2

Name : Avanish Pathak

Section : NA

Roll No : 22515000034

Q1. What is CSS Box Model?

Sol:

The CSS Box Model is a fundamental concept that describes how every HTML element is treated as a rectangular box on a web page. It defines how the size, spacing, and layout of an element are calculated.

The box model consists of four parts, from innermost to outermost:

- Content: The actual text, image, or other media inside the element.
- Padding: The transparent space between the content and the border.
- Border: A line that wraps around the padding and content.
- Margin: The transparent space outside the border, separating the element from other elements.

The total width of an element is calculated as: width + left padding + right padding + left border + right border + left margin + right margin (unless box-sizing: border-box is used, in which case padding and border are included inside the specified width).

Q2. What is media query? Explain with a sample code.

Sol:

A media query is a CSS technique used to apply different styles to a web page depending on the characteristics of the device, such as screen width, height, orientation, or resolution. It is the core technique used to create responsive web designs that adapt to desktops, tablets, and mobile phones. Media queries use the @media rule followed by a condition, and the styles inside are applied only when the condition is true.

```
/* Default style for larger screens */
body {
  background-color: lightblue;
  font-size: 18px;
}

/* Applies when screen width is 768px or less (tablets/mobiles) */
@media screen and (max-width: 768px) {
  body {
    background-color: lightyellow;
    font-size: 14px;
  }
}
```

In the example above, the background color and font size change automatically when the browser window becomes 768px wide or narrower, making the layout responsive.

Q3. What are the features of Bootstrap?

Sol:

Bootstrap is a free, open-source front-end framework used for building responsive and mobile-first websites quickly. Its main features are:

- **Responsive Grid System:** A 12-column grid layout that adapts content to different screen sizes.
- **Pre-styled Components:** Ready-made UI elements like navbars, buttons, cards, modals, carousels, and forms.
- **Mobile-First Approach:** Designed to work smoothly on mobile devices first, then scales up to larger screens.
- **Cross-Browser Compatibility:** Works consistently across Chrome, Firefox, Safari, Edge, etc.
- **Customizable:** Allows easy theming and customization using Sass variables.
- **JavaScript Plugins:** Built-in plugins for dropdowns, tooltips, modals, sliders, etc. (via Bootstrap JS/Popper.js).
- **Utility Classes:** A large set of helper classes for spacing, colors, text alignment, display, etc., reducing the need for custom CSS.
- **Consistent Design:** Provides a uniform look and feel across the entire website.
- **Easy to Learn and Use:** Well-documented with a simple class-based syntax.

Q4. Write a example of basic grid structure in Bootstrap?

Sol:

Bootstrap's grid system is based on a container, rows, and columns, and it uses 12 columns to divide the page horizontally. A basic example is shown below:

```
<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>
```

In this example, the container holds a row, and the row is divided into 3 equal columns (col-md-4 each), since $4 + 4 + 4 = 12$. On medium and larger screens, all three columns appear side by side, while on smaller screens they automatically stack vertically.

Q5. What is CSS Selectors?

Sol:

A CSS selector is a pattern used to select and target the HTML element(s) that a style rule should apply to. Selectors are the part of a CSS rule that comes before the curly braces. Common types of CSS selectors include:

- **Element/Type Selector:** Selects all elements of a given tag, e.g. `p { }`
- **Class Selector:** Selects elements with a specific class, e.g. `.classname { }`
- **ID Selector:** Selects a single element with a specific id, e.g. `#idname { }`
- **Universal Selector:** Selects all elements, written as `* { }`
- **Group Selector:** Applies the same style to multiple selectors, e.g. `h1, h2, p { }`

- Descendant Selector: Selects elements nested inside another, e.g. `div p { }`
- Pseudo-class Selector: Selects elements in a specific state, e.g. `a:hover { }`
- Attribute Selector: Selects elements based on an attribute, e.g. `input[type="text"] { }`

Q6. What are CSS backgrounds, list the properties?

Sol:

The CSS background properties are used to define the background effects (color, image, position, repetition, etc.) for HTML elements. The main background properties are:

Property	Description
background-color	Sets the background color of an element.
background-image	Sets one or more background images for an element.
background-repeat	Defines whether/how the background image repeats (repeat, no-repeat, repeat-x, repeat-y).
background-position	Sets the starting position of the background image.
background-size	Specifies the size of the background image (e.g. cover, contain).
background-attachment	Defines whether the background scrolls with the page or stays fixed.
background-origin	Specifies the positioning area of the background image.
background-clip	Defines how far the background extends within the element (border-box, padding-box, content-box).
background	A shorthand property to set all background properties in one line.

Q7. How can we overlap elements in CSS?

Sol:

Elements can be made to overlap in CSS mainly by using the position property along with the z-index property:

- Positioning: Setting position: absolute, relative, or fixed on elements allows them to be placed on top of one another using the top, left, right, and bottom properties.
- z-index: When elements overlap, z-index controls the stacking order — an element with a higher z-index value appears above elements with a lower value.
- Negative Margins: Using negative margin values can also pull an element over another nearby element.

```
.box1 {
  position: absolute;
  top: 20px; left: 20px;
  z-index: 1;
}

.box2 {
  position: absolute;
  top: 40px; left: 40px;
  z-index: 2; /* appears on top of box1 */
}
```

```
}
```

Q8. What are the different positioning properties in CSS?

Sol:

The CSS position property specifies how an element is positioned in the document. There are five main values:

- **Static:** Default value; elements are positioned according to the normal flow of the page. top, left, right, bottom have no effect.
- **Relative:** The element is positioned relative to its normal position, and can be shifted using top, left, right, bottom without affecting surrounding elements.
- **Absolute:** The element is removed from the normal flow and positioned relative to its nearest positioned ancestor (or the document body if none exists).
- **Fixed:** The element is positioned relative to the browser viewport and stays in the same place even when the page is scrolled.
- **Sticky:** A hybrid of relative and fixed — the element behaves as relative until it crosses a specified scroll threshold, after which it becomes fixed.

Q9. What is ECMAScript?

Sol:

ECMAScript (ES) is a standardized scripting language specification, created and maintained by ECMA International (TC39 committee), that defines the rules, syntax, and features which scripting languages must follow. JavaScript is the most well-known and widely used implementation of the ECMAScript standard. Other languages like JScript and ActionScript are also based on ECMAScript. New versions of ECMAScript (such as ES5, ES6/ES2015, ES2016, and so on) are released periodically, introducing new features like let/const, arrow functions, classes, promises, template literals, and modules, which are then adopted by JavaScript engines in browsers.

Q10. What is the difference between var, let and const in JavaScript. Explain Temporal Dead zone?

Sol:

var, let, and const are all used to declare variables in JavaScript, but they differ in scope, hoisting behavior, and re-assignment rules:

Feature	var	let	const
Scope	Function-scoped	Block-scoped	Block-scoped
Re-declaration	Allowed	Not allowed in same scope	Not allowed in same scope
Re-assignment	Allowed	Allowed	Not allowed
Hoisting	Hoisted and initialized with undefined	Hoisted but not initialized (TDZ applies)	Hoisted but not initialized (TDZ applies)
Global object property	Yes (when declared globally)	No	No

Temporal Dead Zone (TDZ):

The Temporal Dead Zone is the period between the start of a block/scope and the point where a `let` or `const` variable is actually declared and initialized in the code. During this period, the variable exists (it has been hoisted) but cannot be accessed — trying to use it results in a `ReferenceError`. This behavior helps catch bugs caused by using variables before they are properly declared, unlike `var`, which would simply return `undefined`.

```
console.log(a); // undefined (var is hoisted and initialized)
var a = 10;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20; // 'b' is in the Temporal Dead Zone until this line
```